

SFRUTTARE IL PRINCIPIO DI LOCALITÀ

SE IO HO TUTTI GLI INDIRIZZI DI MEMORIA DELLA RAM, QUINDI HO DISPONIBILE TUTTO LO SPAZIO DI INDIRIZZAMENTO, IN UNA CERTA FASE DI TEMPO IO ACCEDO SOLO E UNICAMENTE AD UNA SOTTOFORZIONE DI DATI E ISTRUZIONI.

Cosa devo fare per sfruttare la località? Per sfruttare il principio di località temporale cerco di ottenere i dati con cui accedo più frequentemente, vicino al processore;

Cercando di mettere i dati in cache L1, se poi è troppo grande passo alla cache L2.

La mia memoria è in grado di leggere righe e queste righe sono composte da tanti byte.

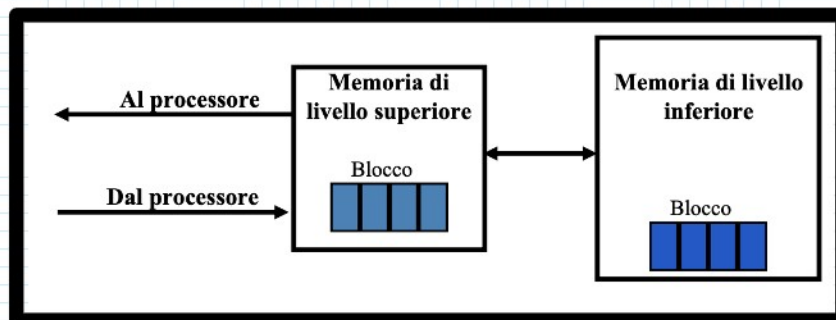
Se io accedo ad un byte e sapendo che esiste la località spaziale, cosa ha senso fare?

Salvare tutta la linea, in un livello di cache superiore;

Cercare di avvicinare più dati possibile sfruttando la località spaziale!

QUINDI I NOSTRI LIVELLI RAGIONANO A BLOCCHI.

UN BLOCCO È UN INSIEME DI BYTE CHE VIENE PRECARICATO OGNI VOLTA CHE VADO A FARE ACCESSO IN MEMORIA.



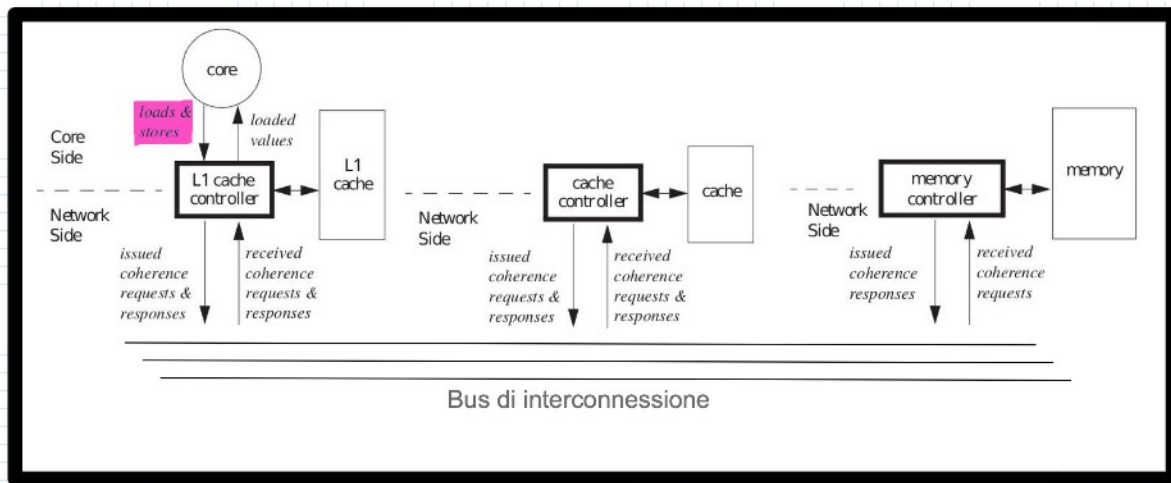
Una volta che ho il blocco in cache L1 da qui gli passerò soltanto il byte che mi interessa; il byte successivo, quando ci verrò a cercare, lo troverò accanto nel record.

Pagherò un accesso che è molto minore in termini di latenza;

SE IL PROCESSORE PARLA SOLTANTO NELLA CACHE DI PRIMO LIVELLO, NON PUÒ ESSERE IN A GESTIRE I LIVELLI INFERIORI.

ABBIAMO VARI CONTROLLORI CHE SONO DELLE MACHINE A STATI IL CUI SCOPO È QUELLO DI GESTIRE LE COPIE DEI DATI NELLE VARIE CACHE E GESTIRE IL TRASFERIMENTO DATI DA UN LIVELLO AD UN LIVELLO INFERIORE.

Cache e controllori della memoria



LA MIA CPU fa delle richieste al controllore della cache di livello L1, LE UNICHE richieste che il processore manda a questo controllore sono richieste **LOAD E STORE**.

Ovvero "dammi un dato", "scrivi il valore ad un certo indirizzo"!

È lui che finisce che se una copia del dato è disponibile in questa cache!

Se è disponibile legge quel dato e lo dà al processore e se non è disponibile è opportuno sapere che tra i vari controllori delle cache esiste un BUS di INTERCONNESSIONE, E chiede a tutti su questo BUS (il controllore della cache L1) chi ha quel dato chiesto dalla CPU! E magari il controllore della cache di secondo livello vede se ce l'ha, se ce l'ha risponde positivamente e gli manda il dato!

E se non ce l'ha si passa al controllore di livello superiore, E COSÌ VIA!

FINO AD ARRIVARE AL CONTROLLORE di memoria, e manderà sull'ADDRESS BUS o sul data bus della memoria, i segnali necessari per recuperare un blocco.

A questo punto la memoria trasferirà questo blocco agli altri controllori, cosicché avendo il controllore della cache L1, possa dare il dato al processore!

↓
Località spaziale: Mi porto più vicino possibile quella area di memoria.

- La cache è strutturata in **linee**
 - Ogni linea contiene un **blocco** (tipicamente 64 byte), ossia la *minima unità di informazione* che viene trasferita tra livelli
- Il processore interagisce soltanto con il **controllore della cache di primo livello**
- Se il dato richiesto si trova nella cache di primo livello, si ha un **cache hit**
- Se il dato richiesto non è presente, si verifica un **cache miss**
 - Il controllore di livello L1 "richiede" al controllore di livello L2 il dato
 - Può verificarsi un hit o un miss nel livello L2
 - In caso di hit in L2, il dato viene copiato in L1
 - In caso di miss in L2, il controllore di livello L2 "richiede" il dato al controllore di livello L3
 - Lo stesso schema si verifica per l'accesso alla RAM

DEFINIZIONI

- **Hit rate** (frequenza di successo): frazione degli accessi in memoria risolti in un dato livello della gerarchia

$$\text{Hit rate} = \text{numero di hit} / \text{numero di accessi a memoria}$$
- **Miss rate** (frequenza di fallimento): $1 - \text{hit rate}$
- **Hit time** (tempo di successo): tempo di accesso al livello in caso di successo
- **Miss penalty** (penalità di fallimento): tempo per trasferire il blocco dal livello inferiore della gerarchia
- **Miss time** (tempo di fallimento): tempo per ottenere l'elemento in caso di miss

$$\text{miss time} = \text{miss penalty} + \text{hit time}$$

CACHE INCLUSIVE / NON INCLUSIVE

Cache inclusive:

- Un livello superiore della gerarchia (più vicino al processore) contiene un sottoinsieme di informazioni dei livelli inferiori
- Tutte le informazioni sono memorizzate nel livello più basso
- Solo il livello massimo di cache (L1 cache) è acceduto direttamente dal processore

Cache non inclusive (o esclusive):

- Alcuni livelli possono essere *non inclusivi*
- Un miss ad un livello superiore (es L1), può comportare l'accesso diretto in memoria
- In questo caso, il dato è scritto solo in L1, ma non in L2
- Se per scrivere in L1 è necessario rimpiazzare un blocco, questo "scende" in L2

Si Adotta questa!

Se ho un miss e vado in memoria, il dato verrà copiato solo in cache L1.

↳ così riduco la latenza;

Immaginiamo di trovarci nella situazione in cui la cache L1 è piena
 Il processore fa una richiesta di un dato che non è presente in cache L1.

Che succede? Il controller chiede al controller di memoria di prendere quel blocco, il controller di memoria consegna il blocco al controller di memoria di cache L1, e cosa fa il controller di cache L1? Prendiamo un blocco già presente in L1 e lo buttiamo via per far spazio al nuovo blocco, e quel vecchio blocco lo scriviamo in L2.

SE DEVO RIMPIAZZARE UN BLOCCO IN L1, QUELLO SCENDE IN L2;

IL DATO UN PO' PIÙ VECCHIO LO COPIO IN L2 ed IN L1 FACCIO SPAZIO AL NUOVO;

Devo tenere traccia di quali blocchi sono disponibili in quali linee.

Non è sufficiente avere all'interno della cache le linee per contenere i blocchi, ma ciascuna linea deve mantenere delle informazioni di controllo.

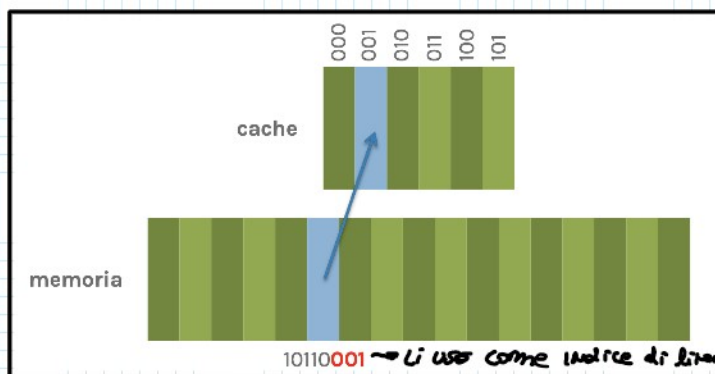
- Una linea di cache non contiene solo il blocco, ma anche **informazioni di controllo** (la *directory*)
- La dimensione in bit di una linea di cache è quindi maggiore della dimensione del blocco stesso
- Le informazioni di controllo consentono di implementare politiche di accesso alle cache più efficienti
- Il *protocollo* di gestione delle cache determina la quantità di informazioni di controllo che sono necessarie per ciascuna linea di cache

MA UN DETERMINATO DATO, IN CHE PUNTO DELLA CACHE PUÒ ESSERE INSERITO?

Come decidiamo in quale linea memorizzare un blocco?

UNA CACHE È PIÙ PICCOLA DELLA MEMORIA.

DOBBIAMO CREARE UN MAPPING TRA TUTTO LO SPAZIO DI INDIRIZZAMENTO E NASARI DI UNA CACHE DI LIVELLO 1 che è più piccola.



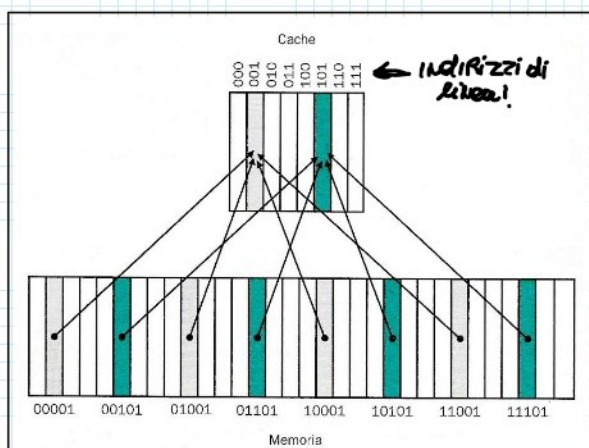
se mi ARRIVA quel blocco, ASSOCIATO a quell'indirizzo, PORTALO sulla cache!

Ogni linea può CONTENERE UN SOLO BLOCCO. L'INDIRIZZO di linea ESISTE!

SE HO 16 GB DI RAM E LA CACHE da 250 Kb, IL CONTROLLORE della CACHE deve ESSERE IN GRADO DI DIRE: "CIASCUN BLOCCO di quei 16 GB DEVE ESSERE ASSOCIATO A UNA O PIÙ LINEE."

- Strategia **direct mapping**: si utilizza una *funzione hash* basata sull'estrazione:
 - esempio: prendo gli n bit meno significativi dell'indirizzo del blocco

PROBLEMA: POSSO AVERE DELLE COLLISIONI!



Abbiamo più blocchi che hanno più bit meno significativi uguali.

NOI DOBBIAMO IDENTIFICARE UN SINGOLO BLOCCO, DOBBIAMO SAPER DISTINGUERE LE VARIE RICHIESTE DEL PROCESSORE.

SOLUZIONE: AGGIUNGO UN'ETICHETTA A CIASCUN BLOCCO

QUINDI CONCRETAMENTE DEVO ANDARE A SCOMPARE L'INDIRIZZO DI MEMORIA, IN VARI PEZZI!

Immaginiamo che il processore voglia leggere un singolo byte in memoria, e più byte sono presenti all'interno dello stesso blocco.

Questi byte contigui all'interno di un blocco avranno gli indirizzi successivi, ci sono due byte successivi all'interno di un singolo blocco.

Posso utilizzare i byte meno significativi di un indirizzo di memoria per rappresentare un offset all'interno di un blocco, se azzerò questi byte, il resto dell'indirizzo può essere utilizzato per identificare l'indice all'interno di un blocco;



Prima abbiamo identificato l'indice utilizzando i 3 bit meno significativi, gli m bit.

Questo n determina la dimensione della mia cache in termini di blocchi.

- L'indirizzo di memoria viene scomposto in tre parti:
 - Tag: i bit più significativi, per discriminare sinonimi
 - Index: bit intermedi, per selezionare una linea di cache
 - Offset: i bit meno significativi per selezionare la parola esatta all'interno del blocco



Immaginiamo che la CPU mi chieda di leggere un certo indirizzo. Come fa il controllore della cache, a sapere che un dato che ha caricato in un certo blocco, è coerente rispetto alla richiesta?

Prendiamo questo indirizzo, estraiamo i bit intermedi, selezioniamo una linea e nella linea ha il tag, che può essere comparato con i byte più significativi dell'indirizzo di memoria che è contenuto nell'accesso attuale, allora sto accedendo a un blocco corretto;

→ se i due tag sono uguali! ✓

Nella directory io mantengo un BIT di stato: = STATE

Se io mi leggo un dato a un indirizzo che è 00000001, leggo 0, Trovo 0, avrei un hit.

MA IN REALTÀ questo blocco non l'ho mai caricato!

Se io mi leggo un dato in un indirizzo che è 00000001, leggo 0, Trovo 0, avrei un HIT.

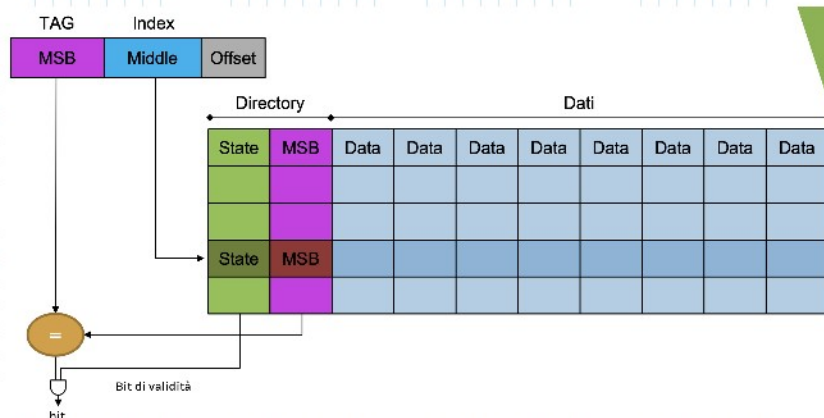
MA IN REALTÀ questo blocco non l'ho mai caricato!

Perché magari ho accesso ora il PC.

→ ecco a cosa mi serve

MI DEVE IDENTIFICARE SE IL BLOCCO È VALIDO!

SE HO 2 INDIRIZZI CHE HANNO LO STESSO INDICE SONO IN UNO STATO DI SINONIMI;



SOLUZIONE: USO LINEE DIVERSE, SE HO TANTI CONFLITTI.

CACHE COMPLETAMENTE ASSOCIATIVA

Immaginiamo io accedo solo ai blocchi grigi della memoria, quanto ho riempito la cache?

Ho riempito un solo blocco. NON VA BENE.

La cache ad accesso diretto continua a conflittare sugli stessi indirizzi!

Cache completamente associativa

- Nelle cache completamente associative (fully associative), ogni blocco può essere memorizzato in qualsiasi linea di cache

Vantaggi:

- utilizzo massimo della cache
- riduzione dei conflitti

Problema: #COMPARAZIONI
" "
#LINEE

Svantaggi:

- Per trovare un blocco, occorre cercarlo in tutta la cache
- La ricerca sequenziale aumenterebbe troppo la latenza
- È necessario cercare in parallelo: si aggiunge un comparatore per ciascuna linea
- Il costo è molto elevato

NON ESISTE PIÙ LA FUNZIONE HASH, ogni volta che carico un nuovo blocco, se è libero, scarto quel blocco all'interno della linea.



MOLTO COSTOSA

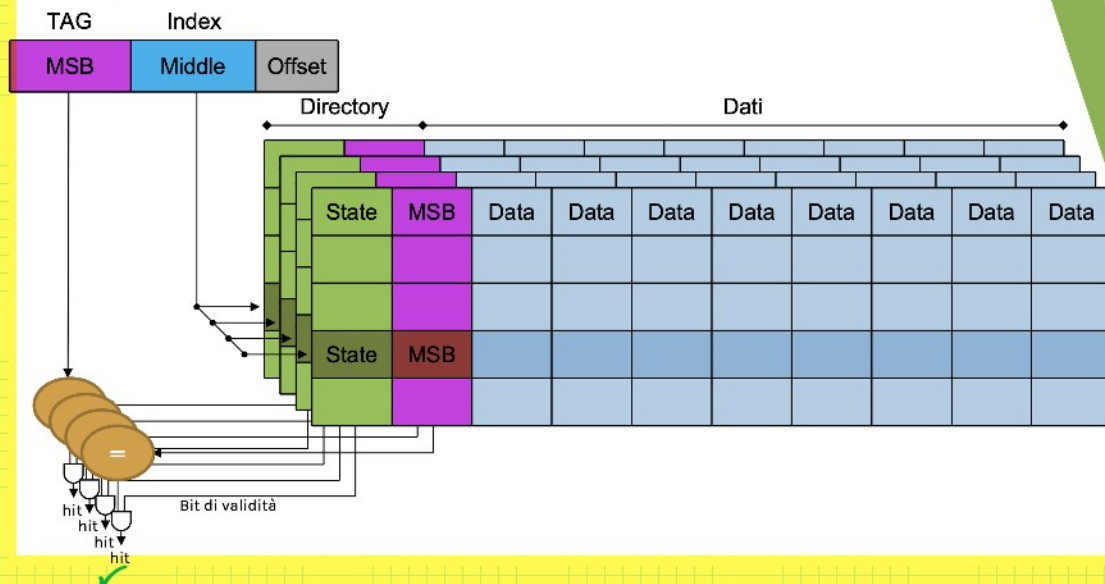
SOLUZIONE FINALE

Cache set associative

Più linee posso contenere un determinato blocco, ma utilizzo tutte quante le linee.
Quindi raggruppo le linee della mia cache, in un insieme

Sfrutto la cache per andare a selezionare l'insieme di linee, all'interno dell'insieme di linee un blocco può andare in qualsiasi linea:

4-way Set Associative Cache: accesso in lettura



↳ l'indice seleziona 4 linee. Un blocco può andare in una di queste 4 linee sovrapposte! e lo metterò in una libera.

LA RICERCA FUNZIONA CON I TAG, E PERÒ HO SOLO 4 COMPARATORI, NON PIÙ COMPARATORI IN OGNI LINEA

↳ per il n° di linee che corrispondono a quell'insieme!

RICORDA CHE L'INDICE CI PERMETTE DI SELEZIONARE UNA LINEA CHE POTREBBE ESSER STATO

caricato un blocco!

↳ Utilizziamo un tag e un comparatore!